Jacobian of the bistatic measurement model

This note derives the Jacobian used in `txloc/measurement_model.py` (`BistaticMeasurementModel.jacobian`) and shows the matching code for each step.

The measurement model maps a transmitter state to a bistatic detection:

$$h(\mathbf{x}) = \begin{bmatrix} R \\ \dot{R} \\ \mathrm{az} \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} \mathbf{p}_t \\ \mathbf{v}_t \end{bmatrix}$$

where R is the excess bistatic range, $\dot{R}$ its time derivative, and az the azimuth. The target (aircraft) and receiver states are known inputs. The Jacobian is $H = \partial h / \partial \mathbf{x}$, a 3×6 matrix evaluated at the current transmitter estimate. It is the local linear map from a small transmitter-state error to the induced change in the measurements:

$$h(\mathbf{x} + \delta\mathbf{x}) \approx h(\mathbf{x}) + H\,\delta\mathbf{x}$$

Gradients below are column vectors; each row of H is the transpose of the corresponding gradient.

Notation

Symbol	Meaning
$\mathbf{p}_t$, $\mathbf{v}_t$	transmitter position and velocity (unknown)
$\mathbf{p}_a$, $\mathbf{v}_a$	target aircraft position and velocity (known)
$\mathbf{p}_r$, $\mathbf{v}_r$	receiver position and velocity (known)
$\hat{\mathbf{E}}$, $\hat{\mathbf{N}}$, $\hat{\mathbf{U}}$	east, north, up unit vectors of the local frame at the transmitter ($\hat{\mathbf{U}}$ is the WGS-84 ellipsoid normal)
$\delta(\cdot)$	small change of a quantity

Derived vectors and lengths:

$$\mathbf{a} = \mathbf{p}_a - \mathbf{p}_t, \qquad \mathbf{b} = \mathbf{p}_r - \mathbf{p}_t, \qquad \mathbf{d} = \mathbf{p}_a - \mathbf{p}_r = \mathbf{a} - \mathbf{b}$$

$$r_t = \|\mathbf{a}\|, \qquad l = \|\mathbf{b}\| \text{ (baseline)}, \qquad r_r = \|\mathbf{d}\|$$

$$\mathbf{u}_t = \frac{\mathbf{a}}{r_t}, \qquad \mathbf{u}_r = \frac{\mathbf{d}}{r_r}, \qquad \mathbf{e} = \frac{\mathbf{p}_t - \mathbf{p}_r}{l} = -\frac{\mathbf{b}}{l}$$

Here $\mathbf{a}$ points from transmitter to target, $\mathbf{b}$ from transmitter to receiver, and $\mathbf{d}$ from receiver to target.

Three rules do most of the work:

$$\frac{\partial \mathbf{u}}{\partial \mathbf{a}} = \frac{I - \mathbf{u}\mathbf{u}^\top}{\|\mathbf{a}\|} \quad \text{for } \mathbf{u} = \frac{\mathbf{a}}{\|\mathbf{a}\|}, \qquad \frac{\partial \mathbf{a}}{\partial \mathbf{p}_t} = \frac{\partial \mathbf{b}}{\partial \mathbf{p}_t} = -I, \qquad \frac{\partial \mathbf{d}}{\partial \mathbf{p}_t} = 0$$

1. Excess range

$$R = r_t + r_r - l$$

Term by term, with respect to $\mathbf{p}_t$:

$\nabla r_t = -\mathbf{u}_t$ (the norm's gradient is the unit vector, and $\mathbf{a}$ moves by $-I$), $\nabla r_r = \mathbf{0}$, $\nabla l = +\mathbf{e}$ ($l = \|\mathbf{p}_t -$

Therefore

$$\boxed{\frac{\partial R}{\partial \mathbf{p}_t} = -\mathbf{u}_t - \mathbf{e}, \qquad \frac{\partial R}{\partial \mathbf{v}_t} = \mathbf{0}}$$

2. Range rate

Differentiate each term of R in time, using $\dot{\mathbf{a}} = \mathbf{v}_a - \mathbf{v}_t$, $\dot{\mathbf{b}} = \mathbf{v}_r - \mathbf{v}_t$ and $\dot{\mathbf{d}} = \mathbf{v}_a - \mathbf{v}_r$:

$$\dot{R} = \mathbf{u}_t^\top \mathbf{w}_t + \mathbf{u}_r^\top (\mathbf{v}_a - \mathbf{v}_r) - \mathbf{e}^\top \mathbf{w}_b, \qquad \mathbf{w}_t = \mathbf{v}_a - \mathbf{v}_t, \quad \mathbf{w}_b = \mathbf{v}_t - \mathbf{v}_r$$

Velocity columns. $\mathbf{w}_t$ depends on $\mathbf{v}_t$ with $-I$, and $\mathbf{w}_b$ with $+I$:

$$\boxed{\frac{\partial \dot{R}}{\partial \mathbf{v}_t} = -\mathbf{u}_t - \mathbf{e}} \qquad \text{(the same vector as } \partial R/\partial \mathbf{p}_t)$$

Position columns. Only $\mathbf{u}_t$ and $\mathbf{e}$ depend on $\mathbf{p}_t$. Using the unit-vector rule with $\partial \mathbf{a}/\partial \mathbf{p}_t = -I$, and $\partial \mathbf{e}/\partial \mathbf{p}_t = (I - \mathbf{e}\mathbf{e}^\top)/l$:

$$\boxed{\frac{\partial \dot{R}}{\partial \mathbf{p}_t} = -\frac{(I - \mathbf{u}_t \mathbf{u}_t^\top)\,\mathbf{w}_t}{r_t} - \frac{(I - \mathbf{e}\mathbf{e}^\top)\,\mathbf{w}_b}{l}}$$

The $\mathbf{u}_r$ term drops out because $\mathbf{d}$ does not depend on the transmitter.

Code for sections 1 and 2 (`measurement_model.py`)

```
drdp = -g.u_t - g.e                          # dR/dp_t  and  dRdot/dv_t
w_t = target.vel_ecef_mps - tx.vel_ecef_mps
w_b = tx.vel_ecef_mps - rx.vel_ecef_mps
ratedp = -(_I3 - np.outer(g.u_t, g.u_t)) @ w_t / g.r_t - (
    _I3 - np.outer(g.e, g.e)
) @ w_b / g.baseline_m                       # dRdot/dp_t
```

where `_geometry` supplies `u_t = a / r_t`, `e = -b / baseline`, and so on.

3. Azimuth

The azimuth is the clockwise angle, seen from above at the transmitter, from $\mathbf{b}$ (towards the receiver) to $\mathbf{a}$ (towards the target). The horizontal projections are $\mathbf{a}_h = \mathbf{a} - (\mathbf{a}^\top \hat{\mathbf{U}})\hat{\mathbf{U}}$ and $\mathbf{b}_h = \mathbf{b} - (\mathbf{b}^\top \hat{\mathbf{U}})\hat{\mathbf{U}}$, so

$$\text{az} = \operatorname{atan2}(s,\, c)$$

$$s = \hat{\mathbf{U}}^\top(\mathbf{a}\times\mathbf{b}) = \hat{\mathbf{U}}^\top(\mathbf{a}_h\times\mathbf{b}_h), \qquad c = \mathbf{a}^\top\mathbf{b} - (\mathbf{a}^\top\hat{\mathbf{U}})(\mathbf{b}^\top\hat{\mathbf{U}}) = \mathbf{a}_h^\top\mathbf{b}_h$$

Here s is the sine-like part and c the cosine-like part. Sanity check: $\mathbf{a}$ east, $\mathbf{b}$ north and $\hat{\mathbf{U}}$ up give $s = +1$, $c = 0$, so az $= +90°$, which is clockwise.

Code for the quantities (`_geometry`)

```
az_num = float(up @ np.cross(a, b))          # s
az_den = float(a @ b - au * bu)              # c   (au = a.up, bu = b.up)
```

The derivative of atan2 is

$$\nabla\,\mathrm{az} = \frac{c\,\nabla s - s\,\nabla c}{s^2 + c^2}$$

so ∇s and ∇c with respect to $\mathbf{p}_t$ are needed. Here $\mathbf{a}$, $\mathbf{b}$ and $\hat{\mathbf{U}}$ all depend on $\mathbf{p}_t$. Let $J_U = \partial\hat{\mathbf{U}}/\partial\mathbf{p}_t$ (a symmetric 3×3 matrix, derived in section 4), write $a_U = \mathbf{a}^\top\hat{\mathbf{U}}$ and $b_U = \mathbf{b}^\top\hat{\mathbf{U}}$, and let $\delta\mathbf{p}$ be a small change in $\mathbf{p}_t$, so that $\delta\mathbf{a} = \delta\mathbf{b} = -\delta\mathbf{p}$ and $\delta\hat{\mathbf{U}} = J_U\,\delta\mathbf{p}$.

Numerator.

$$\begin{aligned}
\delta s &= \delta\hat{\mathbf{U}}^\top(\mathbf{a}\times\mathbf{b}) + \hat{\mathbf{U}}^\top(\delta\mathbf{a}\times\mathbf{b}) + \hat{\mathbf{U}}^\top(\mathbf{a}\times\delta\mathbf{b}) \\
&= \delta\mathbf{p}^\top J_U(\mathbf{a}\times\mathbf{b}) - \delta\mathbf{p}^\top(\mathbf{b}\times\hat{\mathbf{U}}) - \delta\mathbf{p}^\top(\hat{\mathbf{U}}\times\mathbf{a}) \\
&= \delta\mathbf{p}^\top\Big[J_U(\mathbf{a}\times\mathbf{b}) + (\mathbf{a}-\mathbf{b})\times\hat{\mathbf{U}}\Big]
\end{aligned}$$

The middle step uses the scalar triple product to move $\delta\mathbf{p}$ to the front, and the symmetry of J_U. Since $\mathbf{a}-\mathbf{b} = \mathbf{d}$:

$$\boxed{\nabla s = \mathbf{d}\times\hat{\mathbf{U}} + J_U\,(\mathbf{a}\times\mathbf{b})}$$

Denominator.

$$\delta(\mathbf{a}^\top\mathbf{b}) = -\delta\mathbf{p}^\top(\mathbf{a}+\mathbf{b})$$

$$\delta(a_U b_U) = -(\delta\mathbf{p}^\top\hat{\mathbf{U}})(a_U+b_U) + b_U\,\delta\mathbf{p}^\top J_U\mathbf{a} + a_U\,\delta\mathbf{p}^\top J_U\mathbf{b}$$

Since $c = \mathbf{a}^\top\mathbf{b} - a_U b_U$:

$$\boxed{\nabla c = -(\mathbf{a}+\mathbf{b}) + (a_U+b_U)\,\hat{\mathbf{U}} - J_U\,(b_U\,\mathbf{a} + a_U\,\mathbf{b})}$$

Code for section 3

```
au, bu = float(g.a @ up), float(g.b @ up)
d_r = target.pos_ecef_m - rx.pos_ecef_m                  # d
g_num = np.cross(d_r, up) + jn @ np.cross(g.a, g.b)      # grad s
g_den = -(g.a + g.b) + (au + bu) * up - jn @ (bu * g.a + au * g.b)   # grad c
azdp = (g.az_den * g_num - g.az_num * g_den) / (g.az_num**2 + g.az_den**2)
```

Azimuth does not depend on $\mathbf{v}_t$, so its velocity columns are zero.

4. How the local vertical tilts: J_U

$\hat{\mathbf{U}}$ is the WGS-84 ellipsoid normal, so it depends only on the transmitter's geodetic latitude φ and longitude λ:

$$\hat{\mathbf{U}} = \begin{bmatrix} \cos\varphi\cos\lambda \\ \cos\varphi\sin\lambda \\ \sin\varphi \end{bmatrix}, \qquad \frac{\partial\hat{\mathbf{U}}}{\partial\varphi} = \hat{\mathbf{N}}, \qquad \frac{\partial\hat{\mathbf{U}}}{\partial\lambda} = \cos\varphi\,\hat{\mathbf{E}}$$

How latitude and longitude respond to a small displacement $\delta\mathbf{p}$ of the point, with h the ellipsoidal height:

$$\delta\varphi = \frac{\hat{\mathbf{N}}^\top\delta\mathbf{p}}{M+h}, \qquad \delta\lambda = \frac{\hat{\mathbf{E}}^\top\delta\mathbf{p}}{(R_n+h)\cos\varphi}$$

Moving straight up leaves $\hat{\mathbf{U}}$ unchanged. The radii of curvature are

$$M = \frac{A\,(1-\varepsilon^2)}{(1-\varepsilon^2\sin^2\varphi)^{3/2}} \quad \text{(meridian)}, \qquad R_n = \frac{A}{\sqrt{1-\varepsilon^2\sin^2\varphi}} \quad \text{(prime vertical)}$$

with A the WGS-84 semi-major axis and ε^2 its first eccentricity squared. Combining the two contributions, $\delta\hat{\mathbf{U}} = \hat{\mathbf{N}}\,\delta\varphi + \cos\varphi\,\hat{\mathbf{E}}\,\delta\lambda$, so

$$\boxed{J_U = \frac{\hat{\mathbf{N}}\hat{\mathbf{N}}^\top}{M+h} + \frac{\hat{\mathbf{E}}\hat{\mathbf{E}}^\top}{R_n+h}}$$

Code (`txloc/geodesy.py`, `ellipsoid_frame`)

```
w = math.sqrt(1.0 - WGS84_E2 * sl * sl)
r_n = WGS84_A / w
r_m = WGS84_A * (1.0 - WGS84_E2) / (w * w * w)
jac = np.outer(north, north) / (r_m + alt) + np.outer(east, east) / (r_n + alt)
```

5. Assembled 3×6 Jacobian

Rows are range, range rate and azimuth; columns are $\partial/\partial\mathbf{p}_t$ and $\partial/\partial\mathbf{v}_t$:

$$H = \begin{bmatrix} -(\mathbf{u}_t+\mathbf{e})^\top & \mathbf{0}^\top \\ \left(\dfrac{\partial\dot{R}}{\partial\mathbf{p}_t}\right)^\top & -(\mathbf{u}_t+\mathbf{e})^\top \\ (\nabla\,\mathrm{az})^\top & \mathbf{0}^\top \end{bmatrix}$$

with $\partial\dot{R}/\partial\mathbf{p}_t$ from section 2 and ∇ az from section 3. In code this is the `rows` dictionary in `jacobian()`:

```
MeasurementComponent.RANGE:      np.concatenate((drdp, np.zeros(3))),
MeasurementComponent.RANGE_RATE: np.concatenate((ratedp, drdp)),
MeasurementComponent.AZIMUTH:    np.concatenate((azdp, np.zeros(3))),
```

Rows are then selected and ordered by the model's `components` setting.

Symbol to code map

Symbol	Code	Where
$\mathbf{a}$, $\mathbf{b}$	`g.a`, `g.b`	`_geometry`
$\mathbf{d}$	`d_r`	`jacobian`
r_t, l, r_r	`g.r_t`, `g.baseline_m`, `g.r_r`	`_geometry`
$\mathbf{u}_t$, $\mathbf{u}_r$, $\mathbf{e}$	`g.u_t`, `g.u_r`, `g.e`	`_geometry`
$\hat{\mathbf{U}}$	`up` (`frame.up`)	`ellipsoid_frame`
J_U	`jn` (`frame.up_jacobian`)	`ellipsoid_frame`
s, c	`g.az_num`, `g.az_den`	`_geometry`
∇s, ∇c	`g_num`, `g_den`	`jacobian`
$\mathbf{w}_t$, $\mathbf{w}_b$	`w_t`, `w_b`	`jacobian`

Verification

Two tests in `tests/test_measurement_model.py` check this against central finite differences:

- `test_jacobian_matches_finite_differences`: the assembled 3×6 matrix.
- `test_ellipsoid_frame_up_jacobian_matches_finite_differences`: J_U on its own, since that piece is the easiest to get subtly wrong.

Both pass.